Python

Basic String

using placeholder

```
x = 10
s = "Hello world %d"%x
```

using formatting

```
x = 12
s = f"Hello world {x}"
```

Logging

```
import logging
import numpy as np

logging.basicConfig(filename='logfile.log',
                            format='%(asctime)s %(levelname)s: %(message)s',
                            filemode='w')

logger = logging.setLogger()
logger.setLevel(logging.DEBUG)
x=np.random.random((2,3,4))
logger.debug(f'x :{x}')
logger.info("Just an information")
logger.warning("Its a warning")
logger.error("Did you try to divide by zero?")
logger.critical("Internet is down")
```

Creating a matrix

```
mat1 = [[0]*n_cols]*n_rows
# or
mat2 = [[0]*n_cols for _ in range(n_rows)]
```

Numpy

`np.random.rand(2,3,4)` -> standard uniform

`np.random.random((2,3))` -> random_sample from standard uniform

`np.random.uniform(0,5,(2,3))` -> non-standard uniform distribution

`np.random.randn(2,3,4)` -> standard normal

`np.random.normal(0,5,(2,3))` -> non-standard normal distribution

```
$a1 = np.random.rand(5,10,2)
$b1 = np.random.rand(5,5,2)
$np.hstack((a1,b1)).shape
(5,15,2)
$np.concatenate((a1,b1),axis=1)
a2 = np.random.rand(10,5,2)
b2 = np.random.rand(5,5,2)
$np.vstack((a2,b2)).shape
(15,5,2)
$np.concatenate((a2,b2),axis=0).shape
(15,5,2)
```

`np.nonzero(J)` -> returns tuples of indices of J that are nonzero

`np.maximum(x1, x2)` -> returns he maximum of *x1* and *x2*, element-wise. This is a scalar if both *x1* and *x2* are scalars.

`np.max(J, axis=1)` -> Maximum of *a*. If *axis* is None, the result is a scalar value. If *axis* is an int, the result is an array of dimension `a.ndim - 1`. If *axis* is a tuple, the result is an array of dimension `a.ndim - len(axis)`.

`np.where(J>t, J, 0)` -> fills the position where the boolean condition is true with elements of J, 0 otherwise.

Get indices of N maximum values in a 2D array: example: used in NMS in harris corner detection.

```
n=5
arr=np.random.rand(2,3)
x, y = np.unravel_index(np.argsort(arr.ravel(), axis=None)[::-1][:n], arr.shape)
```

Adding dimension

1. np.newaxis

```
p = np.random.rand(3, 4)
x = p[np.newaxis, :]
```

2.
```
p = np.random.rand(3, 4)
x = p[None, :]
```

Creating a grid using outer product and `np.meshgrid` for grid indices

Code:

```
# Create a grid-like structure using outer product.
# Used in the implementation of positional encoding
# in Transformers.
a = np.arange(5)
b = np.arange(5)
c = np.arange(5, 0, -1)

grid1 = a[:, np.newaxis] @ b[np.newaxis, :]
grid2 = a[:, np.newaxis] @ c[np.newaxis, :]
print(grid1)
print(grid2)

x, y = np.meshgrid(a, b, indexing='ij')
for (i, j) in zip(x, y):
    print("i: ", i, ", j: ", j)
```

Output:

```
[[ 0  0  0  0  0]
 [ 0  1  2  3  4]
 [ 0  2  4  6  8]
 [ 0  3  6  9 12]
 [ 0  4  8 12 16]]
[[ 0  0  0  0  0]
 [ 5  4  3  2  1]
 [10  8  6  4  2]
 [15 12  9  6  3]
 [20 16 12  8  4]]
i:  [0 0 0 0 0] , j:  [0 1 2 3 4]
i:  [1 1 1 1 1] , j:  [0 1 2 3 4]
i:  [2 2 2 2 2] , j:  [0 1 2 3 4]
i:  [3 3 3 3 3] , j:  [0 1 2 3 4]
i:  [4 4 4 4 4] , j:  [0 1 2 3 4]
```

Slicing and Indexing in Numpy

Example: numpy_slicing_and_indexing.py

1. Code:

```
# -*- coding: utf-8 -*-
"""
Created on Tue Mar 31 15:12:13 2026

@author: reina
"""

import numpy as np


x = np.random.randint(0, 10, (3, 4, 5))
print(x.shape)
print(x)
print('\n')
# l = [i for i in x]
print([i for i in x])  # List comprehension
print('\n')
# The order returned in list comprehension is
# consistent with the order of priority of indexing
# of arrays in numpy.
print(x[0, :])
print(x[0, :, :])
print('\n')
print(x[1, :])
print(x[1, ...])
print('\n')

print((x[0, :] == x[0, :, :]).all())
print((x[1, :] == x[1, ...]).all())
```

Output:

```
(3, 4, 5)
[[[3 6 7 2 3]
  [0 6 4 5 5]
  [9 4 5 2 0]
  [7 2 4 3 9]]

 [[5 9 9 7 3]
  [9 4 8 4 6]
  [0 7 0 6 6]
  [5 3 9 0 8]]

 [[1 5 7 9 2]
  [5 2 9 4 2]
  [4 8 6 4 2]
  [3 2 7 8 3]]]


[array([[3, 6, 7, 2, 3],
       [0, 6, 4, 5, 5],
       [9, 4, 5, 2, 0],
       [7, 2, 4, 3, 9]], dtype=int32), array([[5, 9, 9, 7, 3],
       [9, 4, 8, 4, 6],
       [0, 7, 0, 6, 6],
       [5, 3, 9, 0, 8]], dtype=int32), array([[1, 5, 7, 9, 2],
       [5, 2, 9, 4, 2],
       [4, 8, 6, 4, 2],
       [3, 2, 7, 8, 3]], dtype=int32)]


[[3 6 7 2 3]
 [0 6 4 5 5]
 [9 4 5 2 0]
 [7 2 4 3 9]]
[[3 6 7 2 3]
 [0 6 4 5 5]
 [9 4 5 2 0]
 [7 2 4 3 9]]


[[5 9 9 7 3]
 [9 4 8 4 6]
 [0 7 0 6 6]
 [5 3 9 0 8]]
[[5 9 9 7 3]
 [9 4 8 4 6]
 [0 7 0 6 6]
 [5 3 9 0 8]]


True
True
```

Slicing and Indexing in Python

1. Code: slicing_and_indexing.py

```python
"""
Slicing in Python:
    a:b:c
a: start
b: end
c: step
"""

a = [1, 2, 3]
b = a
print(a == b)
print(a is b)

c = [1, 2, 3]
d = c.copy()
# Copy a list using :: operator
e = c[::]
print(c == d)
print(c is d)
print(c is e)

# Reverse a list
print(c[::-1])
```

Out:

```
True
True
True
False
```

```
False
[3, 2, 1]
```

Non-Maximum Suppression

1. Code:

```python
# -*- coding: utf-8 -*-
"""
Created on Sat Jul 25 12:18:30 2026

@author: reina
"""

import numpy as np

def non_maximum_suppression(boxes, scores, iou_threshold):
    """
    Apply Non-Maximum Suppression (NMS) to bounding boxes.

    Args:
        boxes: Array-like of shape (N, 4) with boxes in format [x1, y1, x2, y2]
        scores: Array-like of shape (N,) with confidence scores
        iou_threshold: float, IoU threshold for suppression (0 to 1)

    Returns:
        List of indices of kept boxes, ordered by descending score
        Returns -1 for invalid inputs
    """

    # Boxes in x1y1x2y2 format
    if not boxes:
        return []
    boxes = np.array(boxes)
    scores = np.array(scores)
    areas = (boxes[:, 2] - boxes[:, 0]) * (boxes[:, 3] - boxes[:, 1])
    order = scores.argsort()[::-1]  # Descending order
    keep = []

    while len(order) > 0:
        idx = order[0]
        keep.append(idx)

        order = order[1:]
        if len(order) == 0:
            break

        # Compute IOU
        xx1 = np.maximum(boxes[order, 0], boxes[idx, 0])
        yy1 = np.maximum(boxes[order, 1], boxes[idx, 1])
        xx2 = np.minimum(boxes[order, 2], boxes[idx, 2])
        yy2 = np.minimum(boxes[order, 3], boxes[idx, 3])

        w = np.clip(xx2 - xx1, a_min=0, a_max=None)
        h = np.clip(yy2 - yy1, a_min=0, a_max=None)
        inter = w * h
        union = areas[order] + areas[idx] - inter
        iou = inter / union

        # Remove boxes with lower scores but with high overlap with boxes[idx]
        order = order[iou < iou_threshold]

    return np.array(keep)
```

Intersection Over Union (IOU)

1. Code:

```python
# -*- coding: utf-8 -*-
"""
Created on Sat Jul 25 14:37:41 2026

@author: reina
"""

import numpy as np

def iou(box_a, box_b):
    """
    Compute Intersection over Union of two bounding boxes.
    """
    # Write code here

    box_a = np.array(box_a)
    box_b = np.array(box_b)

    # box_a: x1_a, y1_a, x2_a, y2_a
    # box_b: x1_b, y1_b, x2_b, y2_b
```

```
    x1 = np.maximum(box_a[0], box_b[0])
    y1 = np.maximum(box_a[1], box_b[1])
    x2 = np.minimum(box_a[2], box_b[2])
    y2 = np.minimum(box_a[3], box_b[3])

    area_a = (box_a[2] - box_a[0]) * (box_a[3] - box_a[1])
    area_b = (box_b[2] - box_b[0]) * (box_b[3] - box_b[1])

    w = np.clip(x2 - x1, a_min=0, a_max=None)
    h = np.clip(y2 - y1, a_min=0, a_max=None)
    inter = w * h
    union = area_a + area_b - inter
    iou = inter / union

    return iou

box_a = [0, 0, 4, 4]
box_b = [2, 2, 6, 6]
ious = iou(box_a, box_b)
print(ious)
```

Matplotlib

1. `fig.add_subplot`

```
# Plot the images in the batch, along with the corresponding labels
fig = plt.figure(figsize=(10, 4))
# Display 20 images
for idx in np.arange(27):
    ax = fig.add_subplot(3, 9, idx + 1, xticks=[], yticks=[])
    imshow(example_data[idx].detach().numpy())
    ax.set_title(classes[example_targets[idx]])
plt.tight_layout()
plt.show()
```

2. `plt.subplots`

```
# Plot the images in the batch, along withe the corresponding labels
fig, subs = plt.subplots(2, 10, figsize=(25, 4))
for idx, sub in zip(np.arange(20), subs.flatten()):
    sub.imshow(np.squeeze(images[idx]), cmap="gray")
    sub.set_title(str(labels[idx].item()))
    sub.axis("off")
plt.savefig("MNIST-data.png")
plt.show()
```